

Tecnologias e Serviços Multimédia

TRABALHO PRÁTICO Nº 3

DISTRIBUIÇÃO DE ÁUDIO PCM

Mestrado Integrado em Engenharia de Telecomunicações e Informática
Jorge Sousa – a74230 | Leandro Alves – a82157 | J. Eduardo Santos – a82350



UNIVERSIDADE DO MINHO | ANO LETIVO 2019/2020



Índice

Índice	2
Índice de figuras	3
1 Introdução	4
2 Arquitetura	4
3 Servidor	5
3.1 Compressão de áudio	6
3.2 Envio de pacotes	8
4 Gateways.....	9
4.1 Encaminhamento de pacotes	9
4.2 Reenvio de pacotes.....	11
4.3 Comandos do cliente <i>master</i>	13
5 Clientes.....	14
5.1 Descompressão	14
5.2 <i>Buffer</i>	14
5.3 Reprodução.....	14
5.3.1 Escolha de músicas	16
5.3.2 Pausa e retoma	17
5.3.3 Repetição	17
6 Conclusão	18
7 Referências	18

Índice de figuras

Figura 2.1 – Arquitetura do sistema.....	4
Figura 2.2 – Topologia da rede no CORE.	5
Figura 3.1 – Estrutura de cada identificador da lista de músicas enviada.	5
Figura 3.2 – Estrutura dos <i>bytes</i> de uma amostra na <i>stream</i> de áudio.	6
Figura 3.3 – Estrutura do cabeçalho do pacote.....	7
Figura 3.4 – Pacotes de áudio enviados para os <i>gateways</i>	8
Figura 4.1 – Diagrama de sequência do encaminhamento de pacotes de áudio. .	10
Figura 4.2 – Pacotes de áudio reencaminhados pelo <i>gateway</i> principal.....	11
Figura 4.3 – Reenvio de pacotes de áudio no <i>gateway</i> principal.	13
Figura 5.1 – Recepção e reprodução de pacotes de áudio no cliente.	15
Figura 5.2 – Lista de músicas e comandos do cliente <i>master</i>	16
Figura 5.3 – Estrutura do comando de pausa/resumo.	17

1 Introdução

Neste trabalho prático foi implementado um sistema de distribuição de áudio numa rede com eventuais limitações de ritmo binário disponível e latência estimada, constituída por grupos de clientes que armazenam e reproduzem pacotes de áudio recebidos.

Neste relatório encontram-se explicados a arquitetura implementada e os métodos de compressão, envio, reenvio, descompressão, armazenamento e reprodução dos pacotes de áudio recebidos; assim como métodos de controlo das músicas reproduzidas.

2 Arquitetura

A arquitetura deste sistema contém um servidor, que armazena ficheiros de áudio, conectado via TCP (*Transmission Control Protocol*) por *gateways* para encaminhar os pacotes de áudio recebidos pelo servidor, e clientes que reproduzem os pacotes recebidos.

A topologia usada encontra-se na Figura 2.1 e Figura 2.2 e foi desenvolvida no *software* de emulação de redes CORE [1].

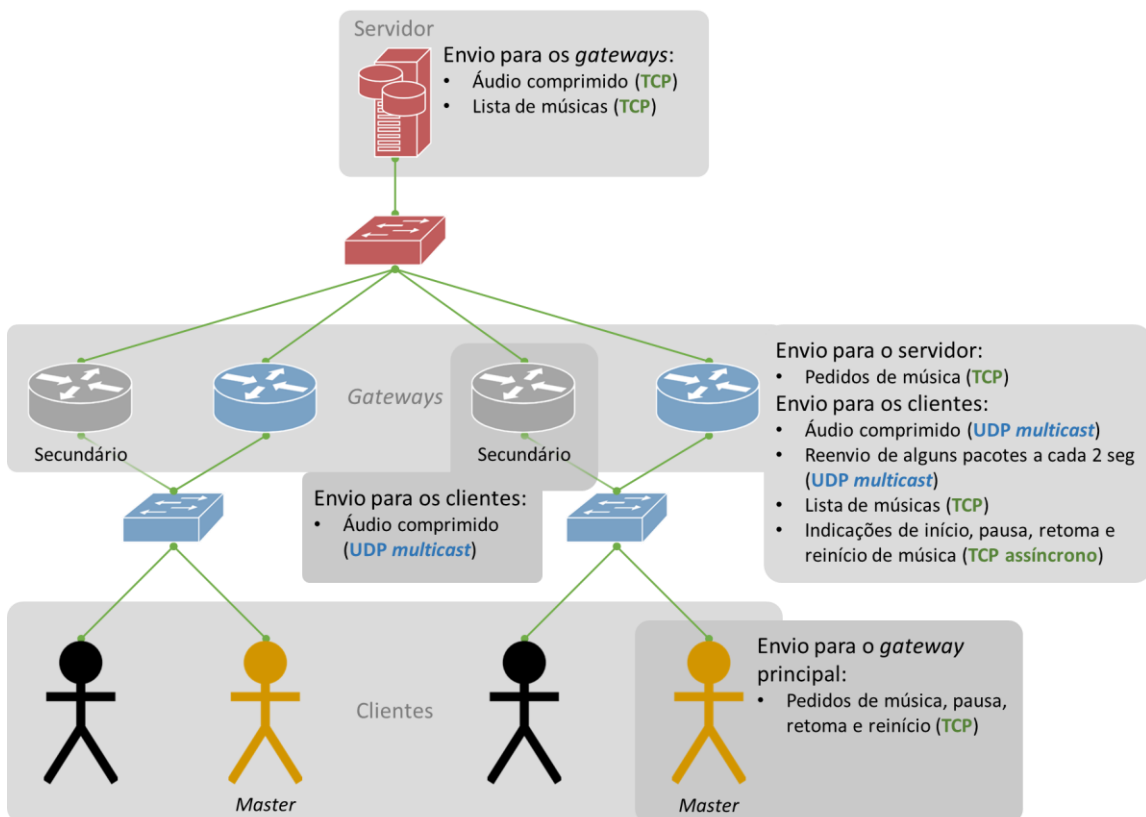


Figura 2.1 – Arquitetura do sistema.

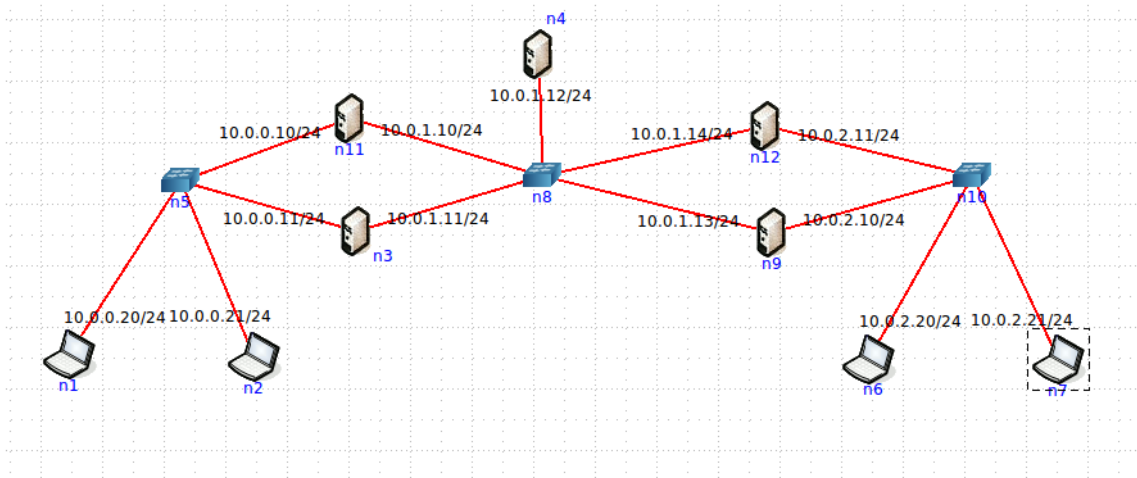


Figura 2.2 – Topologia da rede no CORE.

3 Servidor

Na arquitetura, o papel do servidor é de servir os *gateways* com conexões TCP em *multi-thread*, sendo para cada par criada uma *thread* para serem escutados todos os *gateways* principais assincronamente.

O servidor também é responsável por armazenar todos os ficheiros de áudio. Sempre que há uma conexão de um par de *gateways*, o servidor envia ao principal a lista desses ficheiros, sendo cada um identificado pelos campos da Figura 3.1.

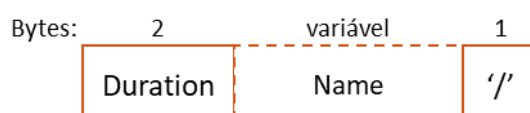


Figura 3.1 – Estrutura de cada identificador da lista de músicas enviada.

- **Duration** – duração da música em segundos;
- **Name** – nome do ficheiro;
- **'/'** – limitador de cada música da lista (foi escolhido o caracter "/" por não poder ser usado em nomes de ficheiros em Unix).

Os ficheiros de áudio compatíveis devem ter as seguintes características:

- Tipo: WAV (PCM (*Pulse-code Modulation*))
- Frequência de amostragem: 44,1 kHz;
- Tamanho de amostra: 16 bits;
- Canais: 2 (*stereo*)
- *Endianness*: *little endian* (*byte* menos significativo armazenado antes do mais significativo).

Todas as amostras encontram-se ordenadas, ocupando na *stream* 4 bytes (16 bits × 2 canais), com a estrutura da Figura 3.2.

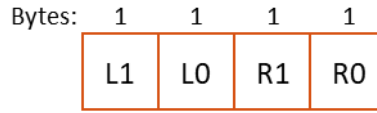


Figura 3.2 – Estrutura dos *bytes* de uma amostra na *stream* de áudio.

- **L1** – *byte* menos significativo do canal esquerdo da amostra;
- **L0** – *byte* mais significativo do canal esquerdo da amostra;
- **R1** – *byte* menos significativo do canal direito da amostra;
- **R0** – *byte* mais significativo do canal direito da amostra.

Por exemplo, um excerto contendo os *bytes* 01 52 A3 F4 contém uma amostra com 5201 no canal esquerdo e F4A3 no canal direito.

Quando um *gateway* principal pede uma música ao servidor (a pedido do cliente *master*), este cria uma *thread* em paralelo e começa a comprimir os pacotes para serem enviados para o respetivo par de *gateways*.

O tamanho original de cada pacote é de 8820 bytes para que haja um número inteiro de pacotes por segundo no ritmo da reprodução, de modo a facilitar os cálculos no reenvio de pacotes no *gateway* principal.

$$R = \frac{L_s \times F_s}{L_{packet}} = \frac{4 \times 44100}{8820} = 20 \text{ packets/s}$$

3.1 Compressão de áudio

Como a mudança dos valores entre amostras é feita gradualmente, a diferença dos valores em amostras adjacentes é normalmente baixa. Este facto ajuda na compressão da *stream* de áudio, sendo usadas na *stream* comprimida essas diferenças em vez dos valores de cada amostra.

Para obter estas diferenças, primeiro é feita a média entre os valores dos dois canais de uma amostra e calculada a diferença entre o canal esquerdo e essa média (*offset*).

$$avg = \frac{L + R}{2}$$

$$ofs = L - avg$$

Para as amostras seguintes é seguido o mesmo processo, sendo também calculadas as diferenças entre as médias e *offsets* da amostra com a anterior (deltas).

$$\Delta avg_i = avg_{i+1} - avg_i$$

$$\Delta ofs_i = ofs_{i+1} - ofs_i$$

Normalmente, estes deltas necessitam de muitos menos *bits* de armazenamento comparando com os valores das amostras.

Como um delta trata-se da diferença entre dois valores de 16 bits, podem ter valores entre 0 e 65535, no caso da amostra, necessitando de até 16 bits. Contudo, se a amostra tiver médias/*offsets* superiores aos da amostra anterior, o delta pode tomar valor negativo, entre -65536 e 1, necessitando de um *bit* adicional para o sinal, estando acumulados 17 bits. Como as médias podem tomar valores não inteiros quando a paridade de cada canal difere, é necessário mais um *bit* para diferir o dígito decimal $-,5$ de $-,0$. Como este dígito no *offset* é sempre igual ao da média, basta acrescentar um *bit* ao delta da média, atingindo 18 bits.

Sabe-se então que no máximo, o delta da média pode chegar aos 18 bits, e o do *offset* aos 17 bits, mas como referido, estas ocasiões são muito raras.

Em cada pacote, todos os campos dos deltas devem ter o mesmo tamanho, sendo os tamanhos atribuídos à média e ao *offset* definidos no cabeçalho. Para que estes valores caibam em apenas 4 bits cada um, o valor para a média é subtraído por 3 e para o *offset* é subtraído por 2. Isto implica que os tamanhos mínimos para os deltas das médias e *offsets* sejam 3 e 2, respetivamente.

Para decidir o tamanho, todos os valores dos deltas da média do pacote são analisados de modo a obter o delta máximo; o mesmo é feito para os deltas do *offset*. Para estes máximos, é calculado o número de *bits* mínimo necessário para armazenar cada um, sendo estes números usados para todos os deltas do pacote e colocados no cabeçalho.

A primeira amostra inicial não é usada no cálculo dos máximos pois o delta seria sempre calculado com a amostra anterior como valor zero, gerando qualquer tamanho possível, com a mesma probabilidade para cada um. Usar a última amostra do pacote anterior também foi colocado fora de questão devido à possibilidade de perda desse pacote no lado do descompressor. Para que o descompressor saiba os valores da primeira amostra, os mesmos são colocados no cabeçalho.

O cabeçalho do pacote tem a estrutura da Figura 3.3.

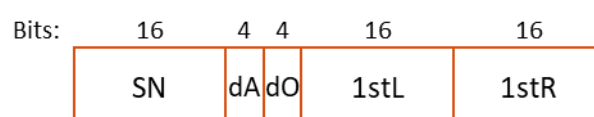


Figura 3.3 – Estrutura do cabeçalho do pacote.

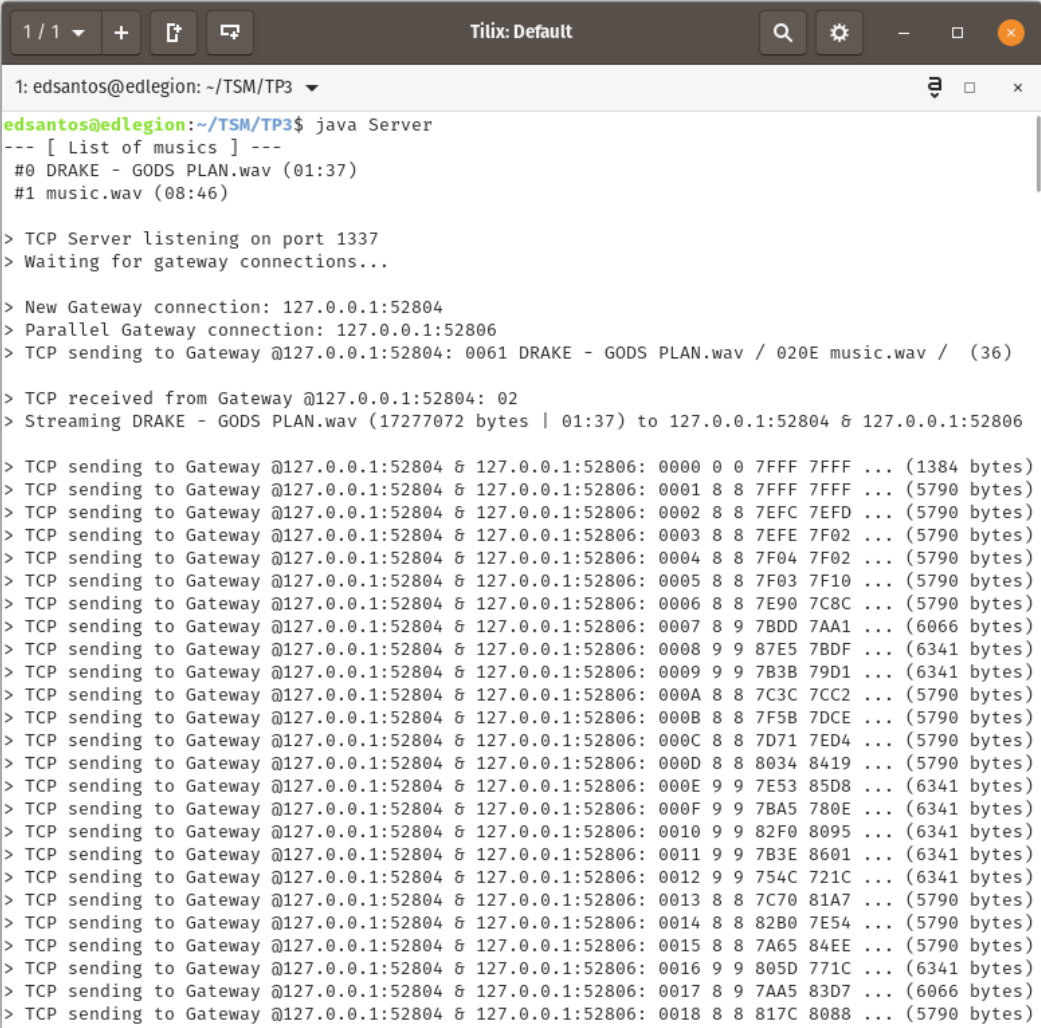
- **SN** (*Sequence Number*) – número de sequência do pacote;
- **dA** – tamanho dos campos dos deltas das médias (subtraído por 3);
- **dO** – tamanho dos campos dos deltas dos *offsets* (subtraído por 2);
- **1stL** – valor do canal esquerdo da primeira amostra;
- **1stR** – valor do canal direito da primeira amostra.

3.2 Envio de pacotes

Depois de comprimido, o pacote é enviado para os dois *sockets* do par de *gateways* cujo principal fez o pedido da respetiva música via TCP (assincronamente via *ConcurrentQueuedList*).

Como o servidor pode não se encontrar na rede local dos *gateways*, a ligação pode não ser tão fiável, podendo TCP ser preferível a UDP (*User Datagram Protocol*).

O conteúdo do cabeçalho dos pacotes enviados e o tamanho dos mesmos é apresentado no servidor, como demonstrado na Figura 3.4.



```
1: edsantos@edlegion: ~/TSM/TP3
edsantos@edlegion:~/TSM/TP3$ java Server
--- [ List of musics ] ---
#0 DRAKE - GODS PLAN.wav (01:37)
#1 music.wav (08:46)

> TCP Server listening on port 1337
> Waiting for gateway connections...

> New Gateway connection: 127.0.0.1:52804
> Parallel Gateway connection: 127.0.0.1:52806
> TCP sending to Gateway @127.0.0.1:52804: 0061 DRAKE - GODS PLAN.wav / 020E music.wav / (36)

> TCP received from Gateway @127.0.0.1:52804: 02
> Streaming DRAKE - GODS PLAN.wav (17277072 bytes | 01:37) to 127.0.0.1:52804 & 127.0.0.1:52806

> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0000 0 0 7FFF 7FFF ... (1384 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0001 8 8 7FFF 7FFF ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0002 8 8 7EFC 7EFD ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0003 8 8 7EFE 7F02 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0004 8 8 7F04 7F02 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0005 8 8 7F03 7F10 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0006 8 8 7E90 7C8C ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0007 8 9 7BDD 7AA1 ... (6066 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0008 9 9 87E5 7BDF ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0009 9 9 7B3B 79D1 ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 000A 8 8 7C3C 7CC2 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 000B 8 8 7F5B 7DCE ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 000C 8 8 7D71 7ED4 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 000D 8 8 8034 8419 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 000E 9 9 7E53 85D8 ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 000F 9 9 7BA5 780E ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0010 9 9 82F0 8095 ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0011 9 9 7B3E 8601 ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0012 9 9 754C 721C ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0013 8 8 7C70 81A7 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0014 8 8 82B0 7E54 ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0015 8 8 7A65 84EE ... (5790 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0016 9 9 805D 771C ... (6341 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0017 8 9 7AA5 83D7 ... (6066 bytes)
> TCP sending to Gateway @127.0.0.1:52804 & 127.0.0.1:52806: 0018 8 8 817C 8088 ... (5790 bytes)
```

Figura 3.4 – Pacotes de áudio enviados para os *gateways*.

Como se pode verificar, o tamanho dos pacotes é consideravelmente menor do que o tamanho original de 8820.

Um dos problemas que encontrámos em sistemas operativos Linux foi que se não houvesse um intervalo de tempo considerável entre cada pacote enviado, alguns dos pacotes recebidos nos *gateways* seriam concatenados devido ao facto do TCP ser um protocolo orientado ao *streaming* e não ao pacote, sendo lida uma *stream* e não um pacote, havendo a possibilidade de essa *stream* conter mais do que um pacote. Como os pacotes têm tamanhos variáveis, não foi possível arranjar um tamanho fixo de leitura de cada pacote recebido. Uma solução para este problema seria indicar esse tamanho no cabeçalho do pacote (ou do anterior), contudo preferimos poupar no tamanho dos pacotes. Outra solução foi de processar os pacotes recebidos assincronamente numa *thread* à parte de modo a dar tempo ao *gateway* de voltar imediatamente à escuta de novos pacotes, contudo isso poderia causar problemas no desempenho devido à possível inundação de *threads*. Em vez disso, optamos por aplicar um *delay* de 40 ms entre cada pacote enviado para não haver tempo de um segundo pacote ser adicionado à *stream* antes do primeiro ser lido pelos *gateways*.

Em Windows, nunca surgiu este problema, sendo permitidos *delays* de 10 ms ou talvez menos.

4 Gateways

Cada grupo contém dois *gateways*, um principal e um secundário. Ambos são clientes TCP do servidor principal, e o *gateway* principal tem um papel adicional de servidor TCP *multi-thread* para os clientes na sua rede local.

O *gateway* é responsável pelo encaminhamento dos pacotes recebidos do servidor para os clientes, e no caso do *gateway* principal, de reenviar alguns pacotes para os mesmos, de informar o servidor da música pedida pelo cliente *master*, e de informar os clientes dos comandos de controlo invocados pelo *master*.

A presença de dois *gateways* aumenta a fiabilidade da receção de pacotes dos clientes, visto que têm duas fontes de receção.

4.1 Encaminhamento de pacotes

Quando é recebido um pacote do servidor, este é reencaminhado via UDP em *multicast* para os clientes na sua rede local. No *gateway* principal, o pacote é também armazenado em *buffer*.

Como os clientes geralmente pertencem à rede local do *gateway*, a ligação é mais fiável, sendo UDP o protocolo mais indicado.

Este fluxo encontra-se esquematizado na Figura 4.1 e demonstrado na Figura 4.2.

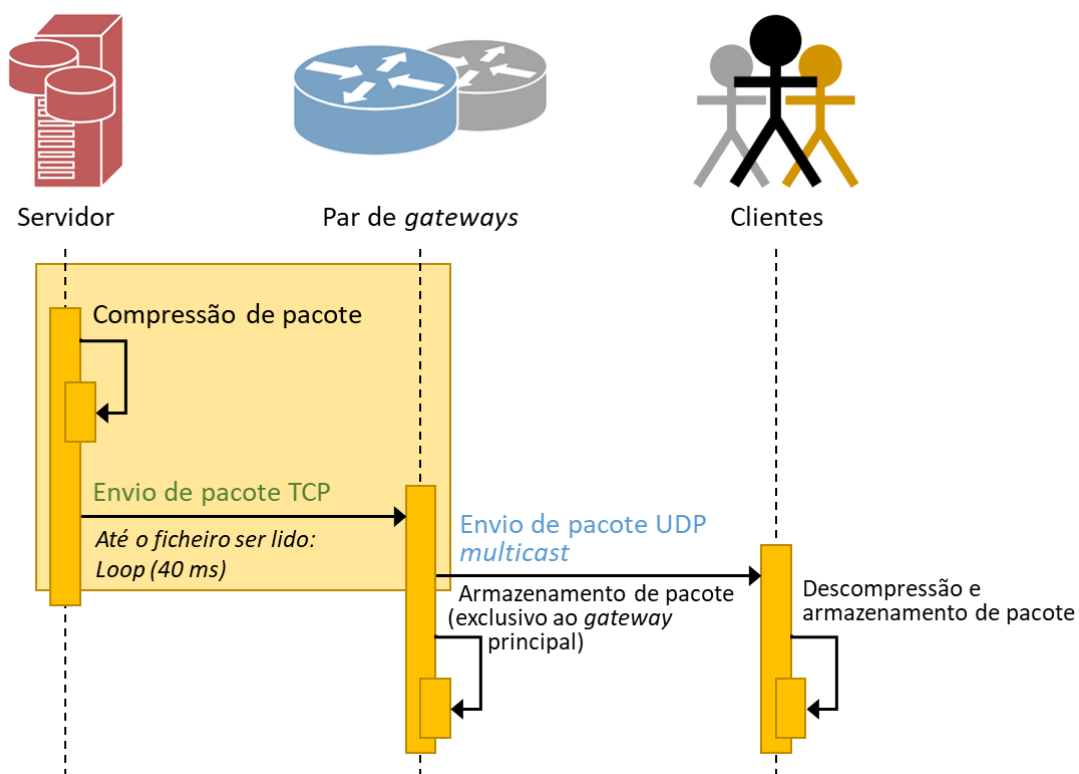


Figura 4.1 – Diagrama de sequência do encaminhamento de pacotes de áudio.

```
1: edsantos@edlegion: ~/TSM/TP3
edsantos@edlegion:~/TSM/TP3$ java Gateway
> Connected to TCP Server @127.0.0.1:1337
> TCP received from Server (36)

> TCP Gateway server listening on port 5000
> Waiting for client connections...

> New client connection: 127.0.0.1:54420
> TCP sending to client @127.0.0.1:54420 (36)

> TCP received from Client @127.0.0.1:54420: 02
> Playing new music #2
> TCP sending to Server: 02

> TCP async sending to Clients: 02
> TCP received from Server: 0000 0 0 7FFF 7FFF ... (1384 bytes)
> Setting initial timestamp
> UDP broadcasting to clients: 0000 ... (1384 bytes)
> TCP received from Server: 0001 8 8 7FFF 7FFF ... (5790 bytes)
> UDP broadcasting to clients: 0001 ... (5790 bytes)
> TCP received from Server: 0002 8 8 7EFC 7EFD ... (5790 bytes)
> UDP broadcasting to clients: 0002 ... (5790 bytes)
> TCP received from Server: 0003 8 8 7EFE 7F02 ... (5790 bytes)
> UDP broadcasting to clients: 0003 ... (5790 bytes)
> TCP received from Server: 0004 8 8 7F04 7F02 ... (5790 bytes)
> UDP broadcasting to clients: 0004 ... (5790 bytes)
> TCP received from Server: 0005 8 8 7F03 7F10 ... (5790 bytes)
> UDP broadcasting to clients: 0005 ... (5790 bytes)
> TCP received from Server: 0006 8 8 7E90 7C8C ... (5790 bytes)
> UDP broadcasting to clients: 0006 ... (5790 bytes)
> TCP received from Server: 0007 8 9 7BDD 7AA1 ... (6066 bytes)
> UDP broadcasting to clients: 0007 ... (6066 bytes)
> TCP received from Server: 0008 9 9 87E5 7BDF ... (6341 bytes)
> UDP broadcasting to clients: 0008 ... (6341 bytes)
> TCP received from Server: 0009 9 9 7B3B 79D1 ... (6341 bytes)
> UDP broadcasting to clients: 0009 ... (6341 bytes)
> TCP received from Server: 000A 8 8 7C3C 7CC2 ... (5790 bytes)
> UDP broadcasting to clients: 000A ... (5790 bytes)
> TCP received from Server: 000B 8 8 7F5B 7DCE ... (5790 bytes)
> UDP broadcasting to clients: 000B ... (5790 bytes)
```

Figura 4.2 – Pacotes de áudio reencaminhados pelo *gateway* principal.

4.2 Reenvio de pacotes

Como UDP não é infalível, pode haver casos de pacotes que não chegam ou não são aceites pelos clientes.

Para minimizar este problema, alguns pacotes armazenados no *buffer* do *gateway* principal são reenviados, numa *thread* à parte (*ScheduledExecutorService*) a cada 2 segundos.

Os pacotes reenviados encontram-se num intervalo dependente da *timestamp* de reprodução atual dos clientes e do último pacote de maior SN armazenado em *buffer*.

Para determinar o pacote do momento atual da reprodução, sabendo que cada segundo corresponde a 20 pacotes, o SN do pacote atual é obtido pela seguinte fórmula:

$$now.sn = (now.timestamp - begin.timestamp) \times 20 + begin.sn$$

- **now.sn** – SN do pacote do momento;
- **now.timestamp** – *timestamp* do momento;
- **begin.timestamp** – *timestamp* inicial (se não tiver havido nenhuma pausa + resumo, é a *timestamp* do início/reinício da reprodução, caso contrário é a *timestamp* do último resumo);
- **begin.sn** – SN do pacote inicial (se não tiver havido nenhuma pausa + resumo, o valor é 0, caso contrário é o SN do momento da pausa/resumo).

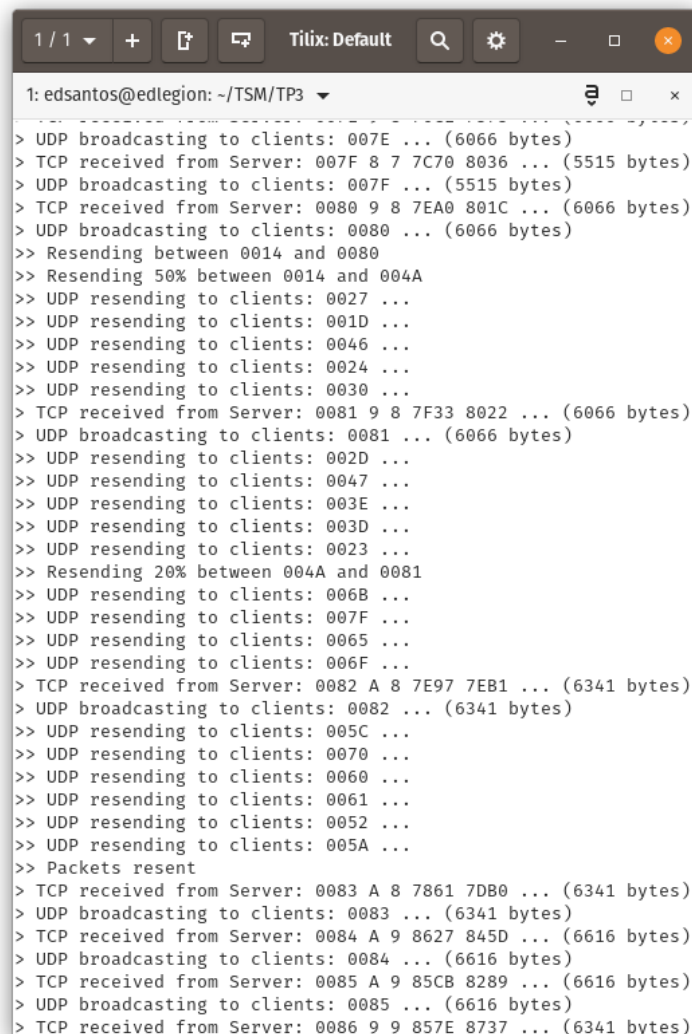
Se a música estiver em pausa, este SN será apenas e sempre o SN do momento da pausa.

O limite mínimo é obtido desta forma porque é inútil o reenvio de pacotes que nunca chegarão a ser reproduzidos devido a estarem atrasados.

Determinados os limites mínimo e máximo dos SN's a reenviar, são aleatoriamente selecionados 50% da primeira metade do *buffer* correspondente a estes limites e 20% da segunda. Não são reenviados todos os pacotes pois há oportunidades para pacotes perdidos de serem reenviados a cada dois segundos e a perda de poucos pacotes muito dispersos não deverá afetar muito a qualidade de reprodução.

Pacotes da primeira metade serão reenviados em maior quantidade do que a segunda pois serão os primeiros a serem reproduzidos, tendo menos oportunidades de reenvio.

Na Figura 4.3 encontra-se um *snapshot* do reenvio de alguns pacotes.



```
1: edsantos@edlegion: ~/TSM/TP3
> UDP broadcasting to clients: 007E ... (6066 bytes)
> TCP received from Server: 007F 8 7 7C70 8036 ... (5515 bytes)
> UDP broadcasting to clients: 007F ... (5515 bytes)
> TCP received from Server: 0080 9 8 7EA0 801C ... (6066 bytes)
> UDP broadcasting to clients: 0080 ... (6066 bytes)
>> Resending between 0014 and 0080
>> Resending 50% between 0014 and 004A
>> UDP resending to clients: 0027 ...
>> UDP resending to clients: 001D ...
>> UDP resending to clients: 0046 ...
>> UDP resending to clients: 0024 ...
>> UDP resending to clients: 0030 ...
> TCP received from Server: 0081 9 8 7F33 8022 ... (6066 bytes)
> UDP broadcasting to clients: 0081 ... (6066 bytes)
>> UDP resending to clients: 002D ...
>> UDP resending to clients: 0047 ...
>> UDP resending to clients: 003E ...
>> UDP resending to clients: 003D ...
>> UDP resending to clients: 0023 ...
>> Resending 20% between 004A and 0081
>> UDP resending to clients: 006B ...
>> UDP resending to clients: 007F ...
>> UDP resending to clients: 0065 ...
>> UDP resending to clients: 006F ...
> TCP received from Server: 0082 A 8 7E97 7EB1 ... (6341 bytes)
> UDP broadcasting to clients: 0082 ... (6341 bytes)
>> UDP resending to clients: 005C ...
>> UDP resending to clients: 0070 ...
>> UDP resending to clients: 0060 ...
>> UDP resending to clients: 0061 ...
>> UDP resending to clients: 0052 ...
>> UDP resending to clients: 005A ...
>> Packets resent
> TCP received from Server: 0083 A 8 7861 7DB0 ... (6341 bytes)
> UDP broadcasting to clients: 0083 ... (6341 bytes)
> TCP received from Server: 0084 A 9 8627 845D ... (6616 bytes)
> UDP broadcasting to clients: 0084 ... (6616 bytes)
> TCP received from Server: 0085 A 9 85CB 8289 ... (6616 bytes)
> UDP broadcasting to clients: 0085 ... (6616 bytes)
> TCP received from Server: 0086 9 9 857E 8737 ... (6341 bytes)
```

Figura 4.3 – Reenvio de pacotes de áudio no *gateway* principal.

4.3 Comandos do cliente *master*

Quando o *gateway* principal recebe um pedido de controlo da reprodução, este reencaminha a mensagem para todos os clientes para que estes adaptem o reprodutor ao que foi pedido.

O comando é recebido em TCP, assim como o reencaminhamento do mesmo para os clientes. Em vez de ser usado o *multicast* como nos pacotes UDP, a mensagem é enviada para cada *socket* conectado ao *gateway* assincronamente através de *wait* e *notifyAll* nas *threads* de cada *socket*.

Os comandos incluem escolha de música, pausa, retoma e repetição.

5 Clientes

Os clientes são responsáveis por reproduzir os pacotes de áudio recebidos do *gateway* e de adaptar essa reprodução consoante as ordens do *master*.

5.1 Descompressão

Ao ser recebido um pacote inexistente no *buffer* (pacote não reenviado), este é descomprimido e armazenado. No caso de receção de um pacote previamente armazenado, este é descartado.

A descompressão é feita a partir do processo inverso à compressão.

Sabendo os tamanhos dos deltas a partir do cabeçalho, os mesmos são retirados do corpo, e a partir dos valores da primeira amostra também no cabeçalho, são calculados as médias e *offsets* das restantes amostras, necessários para obter os valores das próprias amostras.

5.2 Buffer

Depois de um pacote ser descomprimido, este é adicionado a um *buffer* – *array* de pacotes (8820 *bytes*) indexado pelo SN. O SN máximo é também armazenado.

5.3 Reprodução

Quando os clientes recebem a notificação de início de uma nova música, estes ficam inicialmente em pausa, até ordem contrária do *master*.

O *master* lança a ordem de início da reprodução quando o *buffer* contém pelo menos 5 segundos de pacotes.

Quando um cliente recebe o comando de iniciar a reprodução de uma música, é criada uma *thread* onde o sistema de reprodução no *output* de áudio do utilizador é inicializado e o *buffer* é percorrido, de modo a reproduzir cada pacote armazenado.

Para manter o sincronismo entre clientes, quando há um pacote perdido com SN = X , o reprodutor não passa do $X - 1$ para o $X + 1$, em vez disso é assumido que o pacote X tem pacote com os *bytes* todos com valor 0, sendo o mesmo reproduzido, apesar de não emitir som. Este sincronismo serve para que um cliente não se encontre mais adiantado do que os outros na reprodução.

Enquanto a música não estiver terminada, se o último SN reproduzido do *master* atingir o SN máximo armazenado em *buffer*, significa que não há mais pacotes disponíveis para serem reproduzidos. Neste caso, o *master* entra em modo *loading* e envia automaticamente uma ordem de pausa para o *gateway*, de modo a todos os clientes pausarem a reprodução. Este fenómeno foi verificado aplicando um *delay* superior à duração de cada pacote entre cada pacote enviado pelo servidor.

O resumo é ordenado quando forem armazenados pelo menos 5 segundos de pacotes adicionais no *master*.

Na Figura 5.1 encontra-se um *snapshot* da receção e reprodução de pacotes, com o progresso da reprodução representado a vermelho e o de carregamento em *buffer* representado a preto.



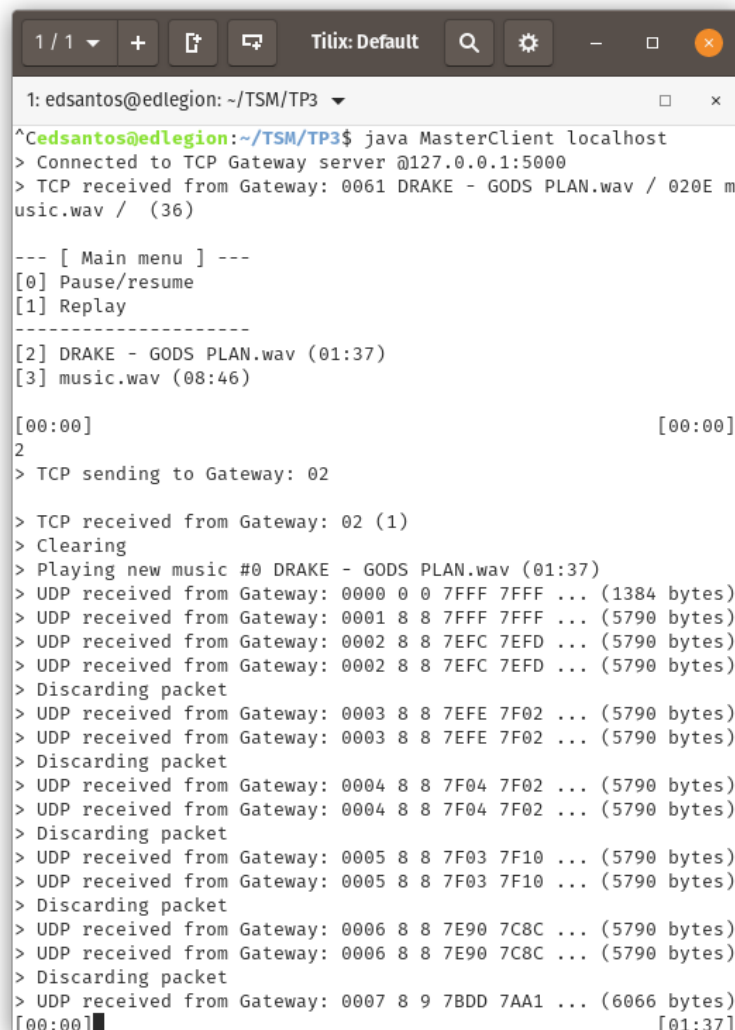
```
1 / 1 + [T] [Q] TiliX: Default [Q] [G] [X]
1: edsantos@edlegion: ~/TSM/TP3
> UDP received from Gateway: 0184 A 9 7603 7991 ... (6616 bytes)
> UDP received from Gateway: 0184 A 9 7603 7991 ... (6616 bytes)
> Discarding packet
> UDP received from Gateway: 0185 A 9 945F 97C2 ... (6616 bytes)
> UDP received from Gateway: 0185 A 9 945F 97C2 ... (6616 bytes)
> Discarding packet
> UDP received from Gateway: 0186 A 9 8A97 7FF0 ... (6616 bytes)
> UDP received from Gateway: 0186 A 9 8A97 7FF0 ... (6616 bytes)
> Discarding packet
> Playing 0104
> Playing 0105
> Playing 0106
> UDP received from Gateway: 0187 9 9 7F61 88B2 ... (6341 bytes)
> UDP received from Gateway: 0187 9 9 7F61 88B2 ... (6341 bytes)
> Discarding packet
> UDP received from Gateway: 0188 A A 8000 79E4 ... (6892 bytes)
> UDP received from Gateway: 0188 A A 8000 79E4 ... (6892 bytes)
> Discarding packet
> UDP received from Gateway: 0189 A B 825C 87FB ... (7167 bytes)
> Playing 0107
> Playing 0108
> Playing 0109
> UDP received from Gateway: 0189 A B 825C 87FB ... (7167 bytes)
> Discarding packet
> UDP received from Gateway: 018A A A 7E00 7974 ... (6892 bytes)
> UDP received from Gateway: 018A A A 7E00 7974 ... (6892 bytes)
> Discarding packet
> UDP received from Gateway: 018B A 9 850D 81AD ... (6616 bytes)
> UDP received from Gateway: 018B A 9 850D 81AD ... (6616 bytes)
> Discarding packet
> UDP received from Gateway: 018C A 9 79B3 61DE ... (6616 bytes)
> Playing 010A
> Playing 010B
> UDP received from Gateway: 018C A 9 79B3 61DE ... (6616 bytes)
> Discarding packet
> UDP received from Gateway: 018D A 9 867C 8DE4 ... (6616 bytes)
> UDP received from Gateway: 018D A 9 867C 8DE4 ... (6616 bytes)
> Discarding packet
[00:13] [01:37]
```

Figura 5.1 – Receção e reprodução de pacotes de áudio no cliente.

Para testar a reprodução em dois clientes em simultâneo, como não foi possível usar mais do que um computador na mesma rede, foi atribuída uma porta diferente ao cliente *slave*, sendo o *multicast* dos *gateways* feito a ambas as portas.

5.3.1 Escolha de músicas

O cliente *master*, a partir da lista de músicas apresentada na Figura 5.2, pode selecionar uma delas enviando um pedido para o *gateway* com o respectivo identificador (≥ 2).



```
1 / 1 + [T] [R] Tilix: Default [Q] [G] [X]
1: edsantos@edlegion: ~/TSM/TP3
^Cedsantos@edlegion:~/TSM/TP3$ java MasterClient localhost
> Connected to TCP Gateway server @127.0.0.1:5000
> TCP received from Gateway: 0061 DRAKE - GODS PLAN.wav / 020E music.wav / (36)

--- [ Main menu ] ---
[0] Pause/resume
[1] Replay
-----
[2] DRAKE - GODS PLAN.wav (01:37)
[3] music.wav (08:46)

[00:00] [00:00]
2
> TCP sending to Gateway: 02

> TCP received from Gateway: 02 (1)
> Clearing
> Playing new music #0 DRAKE - GODS PLAN.wav (01:37)
> UDP received from Gateway: 0000 0 0 7FFF 7FFF ... (1384 bytes)
> UDP received from Gateway: 0001 8 8 7FFF 7FFF ... (5790 bytes)
> UDP received from Gateway: 0002 8 8 7EFC 7EFD ... (5790 bytes)
> UDP received from Gateway: 0002 8 8 7EFC 7EFD ... (5790 bytes)
> Discarding packet
> UDP received from Gateway: 0003 8 8 7EFE 7F02 ... (5790 bytes)
> UDP received from Gateway: 0003 8 8 7EFE 7F02 ... (5790 bytes)
> Discarding packet
> UDP received from Gateway: 0004 8 8 7F04 7F02 ... (5790 bytes)
> UDP received from Gateway: 0004 8 8 7F04 7F02 ... (5790 bytes)
> Discarding packet
> UDP received from Gateway: 0005 8 8 7F03 7F10 ... (5790 bytes)
> UDP received from Gateway: 0005 8 8 7F03 7F10 ... (5790 bytes)
> Discarding packet
> UDP received from Gateway: 0006 8 8 7E90 7C8C ... (5790 bytes)
> UDP received from Gateway: 0006 8 8 7E90 7C8C ... (5790 bytes)
> Discarding packet
> UDP received from Gateway: 0007 8 9 7BDD 7AA1 ... (6066 bytes)
[00:00] [01:37]
```

Figura 5.2 – Lista de músicas e comandos do cliente *master*.

Quando qualquer cliente recebe um identificador de uma música, *buffers* anteriores são esvaziados e a reprodução é pausada até que o *buffer* do *master* esteja suficientemente carregado, havendo uma notificação para tal.

Quando a reprodução de uma música é terminada, o cliente *master* automaticamente envia para o *gateway* um pedido de reprodução da música seguinte da lista (voltando ao início ao chegar ao fim da mesma).

5.3.2 Pausa e retoma

A partir do comando com identificador '0' submetido pelo *master* e recebido pelo *gateway*, os clientes terminam a *thread* de reprodução (para não haver uma espera ativa da mudança de estado e para desligar o sistema de reprodução do *output* de áudio), este comando contém os campos da Figura 5.3.

Bytes:	1	2
	0	SN

Figura 5.3 – Estrutura do comando de pausa/resumo.

- **SN** – SN do momento da pausa.

A partir do mesmo comando, enquanto houver pausa, a reprodução deve ser resumida, fazendo com que os clientes voltem a inicializar a *thread* de reprodução e percorrer o *buffer* a partir do SN do momento da pausa armazenado. Esta mensagem contém os mesmos campos da Figura 5.3.

Como referido anteriormente, o *gateway* principal necessita do SN do momento da pausa para definir o limite inferior da retransmissão de pacotes.

5.3.3 Repetição

Com o comando '1' novamente do cliente *master*, os clientes recebem a ordem de reiniciar a reprodução da mesma música. De modo a aproveitar o *buffer* existente dos clientes e dos *gateways*, o *gateway* não pede ao servidor para reenviar a mesma música.

Os clientes apenas reiniciam a *thread* de reprodução e percorrem o mesmo *buffer* a partir do SN = 0.

6 Conclusão

Neste trabalho prático foram aplicados conhecimentos adquiridos nas aulas práticas acerca da compressão PCM, assim como de outras UC's (Unidade Curriculares), nomeadamente Sistemas Distribuídos para arquiteturas de Cliente/Servidor TCP com mensagens assíncronas e Projeto de Telecomunicações e Informática II para envio e reprodução de pacotes de áudio. Devido ao conhecimento prévio de reprodução e interpretação dos pacotes PCM em ficheiros WAV em Java, foi usada esta linguagem para o desenvolvimento dos programas.

Para além das limitações referidas anteriormente, surgiram outras relativamente à ausência de múltiplos computadores na mesma rede, e na reprodução de áudio em múltiplos no CORE pois não pode haver múltiplos nós a usar o *hardware* do computador *host* para reproduzir áudio. Contudo, verificou-se no CORE que os devidos pacotes chegavam aos clientes, apenas incapazes de serem reproduzidos.

Um mecanismo que apesar de planeado, não foi implementado foi o de adaptação à conexão de clientes a meio da reprodução de uma música, em que pensamos em fazer com que o *gateway* reenviasse todos os pacotes (na mesma em *multicast*) a partir do SN do momento atual da reprodução.

À parte das dificuldades que surgiram e das funcionalidades por implementar, pensamos que concluímos este trabalho prático com sucesso.

7 Referências

- [1] Nrl.navy.mil. n.d. *Common Open Research Emulator (CORE) | Networks and Communication Systems Branch*. [online] Disponível em: <https://www.nrl.navy.mil/itd/ncs/products/core> [Acedido a 11/02/2020].